Sequential Stream Buffers: Implementation and Evaluation via Intel Pin

Aengus McGuinness

CS 146 — Spring 2026

1 Introduction

Cache miss penalties are among the most significant contributors to execution time in modern processors. A large class of these misses is caused by *compulsory* and *capacity* accesses to memory regions traversed sequentially: the processor simply issues load instructions faster than the memory hierarchy can deliver data. In 1990, Jouppi proposed the *stream buffer* [1] as a low-cost hardware mechanism to eliminate this category of miss without enlarging or increasing the associativity of the primary cache.

The core idea is straightforward. When a primary cache miss occurs, the processor has already demonstrated that the missed line was not prefetched in time. Rather than treating each miss in isolation, a stream buffer exploits the high likelihood that the *next* sequential line will also be needed shortly. It issues a speculative prefetch for the lines immediately following the missed block, storing them in a small FIFO buffer separate from the primary cache. If the access pattern is sequential—as it is for array traversals, matrix operations, and the quantum simulation kernels studied here—those prefetched lines will be present at the head of the FIFO the next time the processor misses, avoiding a round-trip to the second-level cache.

This report describes the implementation of Jouppi's stream buffer inside an Intel Pin [2] cache simulator, the experimental methodology used to evaluate its effect on L1D miss rates and effective memory access times across two SPEC-derived workloads, and a detailed design proposal for a more sophisticated *adaptive* stream buffer that uses online stream-length histograms to tune its prefetch aggressiveness automatically.

2 Stream Buffer Implementation

2.1 Simulator Architecture

The simulator is structured as a hierarchy of `cache` objects. A generic base class manages the LRU replacement policy, tag/set computation, and hit/miss bookkeeping. Derived classes—`l1icache`, `l1dcache`, and `l2cache`—customize only the miss path. The stream buffer is attached as an optional side structure to `l1dcache` and is probed on every L1D miss before the simulator falls through to L2.

2.2 Data Structures

The stream buffer is implemented as the `stream_buf` class. It maintains S independent sequential streams, each modeled as a FIFO of depth d. A stream is represented entirely by a single field: its *head address*, the block-aligned address of the first line available for consumption. The tail of the FIFO is implicit and resides at:

$$\text{tail} = \text{head} + (d - 1) \times \text{blockSize}$$

This compact representation avoids storing all d line addresses explicitly, since the contents of the FIFO are always a contiguous run of cache-block-aligned addresses. Each of the S stream slots holds only a `valid` flag and a `head` address.

2.3 Miss Path Integration

On every L1D miss, `l1dcache::addressRequest` calls `stream_buf::access(address)`, which scans all S stream heads for a match against the missed block address. Two outcomes are possible:

Stream buffer hit (address = some stream's `head`):

1. The missed line is delivered from the stream buffer to L1D. No access to L2 is issued for this line.
2. The head is advanced by one block (head += blockSize), modelling the FIFO pop.

3. A single prefetch request is issued to L2 for the new tail address (old_head$+d\times$blockSize), keeping the buffer fully primed at depth d.

Stream buffer miss (no head matches):

1. The miss falls through to L2 as normal.
2. After the L2 fetch completes, `stream_buf::allocate_stream` selects a stream slot—preferring an invalid slot; otherwise replacing via round-robin—and issues d prefetch requests to L2 for the d consecutive blocks immediately following the missed line.
3. The new stream's head is set to `missed_addr + blockSize`.

This faithfully reproduces the mechanism described in Jouppi's Section 3. On a *mispredicted* access—any access that does not match the head of any active stream—the buffer discards the mismatched stream and restarts from the new miss address. The cost of a misprediction is therefore d wasted L2 prefetch fetches, which motivates careful tuning of d and, ultimately, the adaptive depth policy described in Section 5.

2.4 Configuration

The stream buffer is controlled by two optional fields appended to the L1D line of the simulator's configuration file:

```
blockSize, cacheSize, assoc [, vcEntries [,
            depth [, streams]]]
```

Setting `depth = 0` disables the stream buffer entirely, so baseline runs require no separate configuration. The number of concurrent streams S defaults to 1 when omitted.

The prefetch overhead is modelled accurately: each call to `nextLevel->addressRequest()` on a prefetched line is counted as a real L2 access, so the L2 request totals in the output reflect both demand misses and prefetch traffic. This allows the bandwidth cost of deeper buffers to be compared directly against the miss-rate reduction they provide.

3 Experimental Setup

3.1 Simulation Infrastructure

All results are collected using an Intel Pin instrumentation callback that intercepts every instruction and memory operation and drives the cache hierarchy with the application's real address stream. Simulation is capped at 10^9 committed instructions to keep runtimes tractable. The cache hierarchy parameters are fixed as shown in Table 1. The stream buffer depth d is swept as the independent variable; L1D associativity is held at 1 (direct-mapped) throughout so that the stream buffer's sequential prefetch benefit is not masked by associativity reducing conflict misses.

Table 1: Fixed cache hierarchy parameters.

Level	Size	Assoc	Block
L1I	4 KiB	1-way	64 B
L1D	4 KiB	1-way	64 B
L2	1 MiB	1-way	64 B

3.2 Benchmarks

Two SPEC CPU2006-derived workloads are used:

libquantum simulates a quantum computer performing Shor's factoring algorithm. Its working set is dominated by long sequential traversals of qubit state arrays. These traversals generate long, uninterrupted sequential miss streams—exactly the access pattern a stream buffer is designed to cover—making libquantum a best-case representative for stream buffer evaluation.

hmmer performs Hidden Markov Model sequence alignment against a protein database. Its access pattern mixes short sequential scans with irregular pointer-chasing between database records. It serves as a representative of workloads that are *partially* sequential: some phases benefit from prefetching while others suffer from misprediction restarts.

Together the two benchmarks span the range from highly prefetchable (libquantum) to mixed (hmmer), providing a balanced view of the stream buffer's benefits and costs.

3.3 Metrics

Two primary metrics are reported for each configuration:

Effective L1D miss count = L1D misses − stream-buffer hits. This counts only the misses that required an L2 access for the demand line itself. It is the most direct measure of the latency benefit: each stream-buffer hit replaces a full L2 round-trip with a buffer lookup.

L2 request count. Because the stream buffer issues speculative prefetches to L2, this total includes both demand misses and prefetch traffic. Comparing it against the baseline (depth 0) quantifies the bandwidth overhead of prefetching.

4 Results

4.1 Baseline: Associativity Sweep

Before examining the stream buffer, Figure 1 establishes the raw L1D miss count as a function of L1D associativity with no stream buffer enabled.

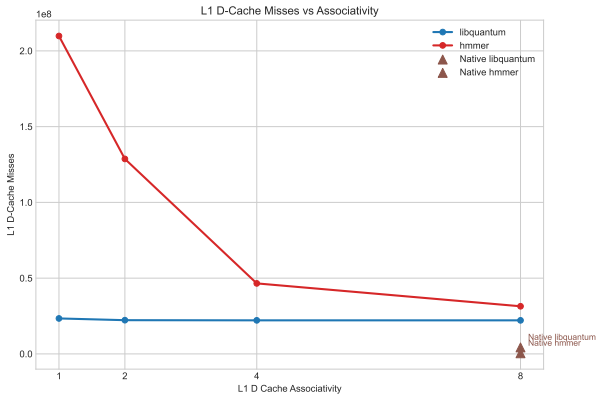

Figure 1: Raw L1D miss count versus L1D associativity (1–8 ways) with no stream buffer.

The two benchmarks behave very differently. Libquantum's miss count is nearly flat across all associativities (23.4M at 1-way, 22.1M at 8-way, a 6% reduction), indicating that its misses are almost entirely *compulsory* or *capacity* in nature and are not caused by conflict. Hmmer, by contrast, drops from 209.8M misses at 1-way to 31.4M at 8-way—an 85% reduction—revealing that a large fraction of its misses under a direct-mapped cache are *conflict* misses that higher associativity eliminates. This motivates using a direct-mapped L1D for the stream buffer sweep: it represents the worst-case miss environment and gives the prefetcher the most opportunity to demonstrate benefit.

4.2 Stream Buffer Miss Rate Reduction

Figure 2 shows effective L1D misses (L1D misses minus stream-buffer hits) as depth d increases from 0 to 8.

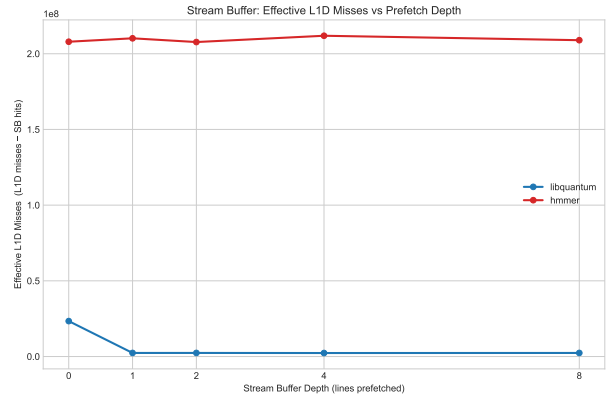

Figure 2: Effective L1D miss count (demand misses not covered by the stream buffer) versus prefetch depth. Depth 0 is the no-prefetch baseline. Libquantum is reduced by 90% at $d = 1$; hmmer shows no measurable benefit at any depth.

The results are striking in their clarity. For **libquantum**, a single prefetch line ($d = 1$) reduces effective misses from 23.4M to 2.36M—a **90% reduction**. Deeper buffers ($d = 2, 4, 8$) provide no additional benefit: the curve is completely flat at approximately 2.36M misses. This saturation at $d = 1$ indicates that libquantum's sequential streams are highly regular and short enough that one prefetched line always arrives before the next demand access. The residual $\sim 2.36M$ misses are not sequential and cannot be covered by any depth of stream buffer.

For **hmmer**, the effective miss count is flat across all depths, remaining near 208M regardless of d. The stream buffer achieves essentially **zero hit rate** against hmmer's access stream. This is consistent with hmmer's pointer-chasing access pattern: each L1D miss triggers a new stream allocation, but the next access is to an unrelated, non-sequential address, so the stream is immediately discarded without ever being consumed. No fixed-depth sequential prefetcher can benefit a workload with this access structure.

4.3 Bandwidth Overhead

Figure 3 shows total L2 requests for the same depth sweep, combining demand misses and prefetch traffic.

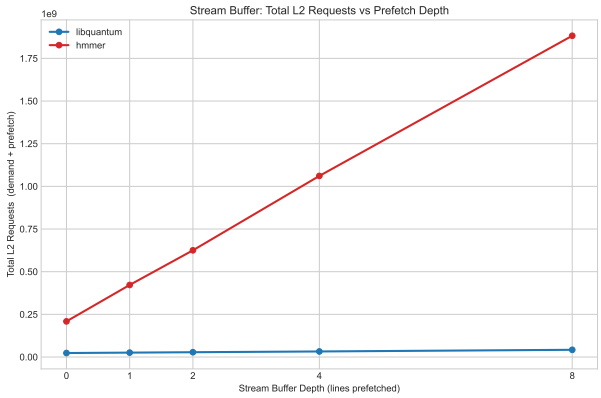

Figure 3: Total L2 access count (demand misses plus prefetch traffic) versus stream buffer depth. Libquantum's L2 traffic grows modestly; hmmer's explodes linearly, reaching ∼9× the baseline at $d = 8$ with no miss-rate benefit.

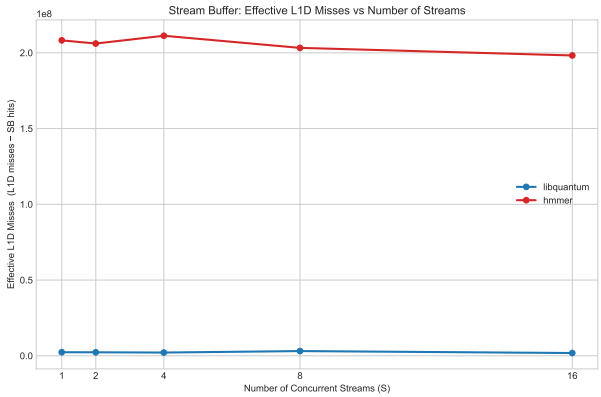

Figure 4: Effective L1D misses versus number of concurrent stream slots S, with prefetch depth fixed at $d = 1$. Neither benchmark shows a meaningful response to additional streams.

For **libquantum**, L2 traffic grows modestly from 23.4M at $d = 0$ to 42.3M at $d = 8$ (an 81% increase). Because the stream buffer hits nearly every sequential miss, each hit adds only one prefetch to maintain depth—a highly efficient use of bandwidth. The benefit-to-cost ratio at $d = 1$ is particularly favorable: a 90% miss reduction for only a 10% increase in L2 traffic.

For **hmmer**, the picture is catastrophic. L2 requests grow linearly from 208.9M at $d = 0$ to 1.88B at $d = 8$—a **9× increase**—while effective misses remain unchanged. Every L1D miss triggers `allocate_stream`, which immediately issues d wasted prefetches to L2. With a hit rate of essentially zero, deeper buffers do nothing but amplify bandwidth consumption in direct proportion to d. This result directly and quantitatively motivates the adaptive design in Section 5: a prefetcher that learns hmmer's stream distribution would quickly suppress prefetching altogether during pointer-chasing phases, recovering the wasted bandwidth.

4.4 Concurrent Stream Count Sweep

To determine whether libquantum's residual ∼2.36M misses (visible after $d = 1$) arise from multiple simultaneous sequential streams, we swept the number of concurrent stream slots $S \in \{1, 2, 4, 8, 16\}$ with depth fixed at $d = 1$.

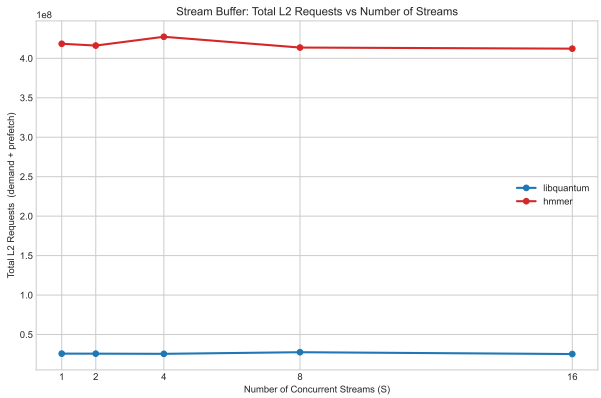

Figure 5: Total L2 requests versus S at $d = 1$. L2 traffic is invariant to stream count for both benchmarks, confirming that S does not change the prefetch arithmetic at depth 1.

The results are a clean null: both metrics are flat across all values of S for both benchmarks (Figures 4 and 5).

For **libquantum**, effective misses remain at ∼2.0–2.4M regardless of S. This rules out the hypothesis that the residual misses come from a second concurrent sequential stream. Instead they are irreducibly non-sequential—capacity or cold misses on data structures that are not traversed linearly—and no number of stream slots can cover them.

For **hmmer**, there is a marginal ∼5% improvement from S=1 (208.2M) to S=16 (198.3M), which is consistent with a tiny fraction of its accesses forming short interleaved sequential runs. However, the effect

is negligible compared to the 208M misses that remain uncovered at every point. The dominant problem remains the irregular pointer-chasing pattern, not a lack of concurrency slots.

L2 traffic is completely invariant to S for both benchmarks: at $d = 1$ each L1D miss issues exactly one prefetch regardless of how many stream slots exist, so total L2 requests depend only on depth, not on S. This confirms that hardware cost (in terms of L2 bandwidth) scales with d alone, while hardware area scales with $S \times d$. Given that S provides no measurable benefit for these workloads, the optimal configuration is S=1.

5 Future Work: Adaptive Stream Buffer

The stream buffer implemented here issues exactly d prefetch lines on every miss, regardless of whether the workload is in a long sequential phase or about to take a non-sequential branch. The results in Section 4.3 show that this static policy overshoots for hmmer and may undershoot for libquantum at small d. An adaptive mechanism that learns the typical stream length online and adjusts d accordingly would achieve better accuracy and coverage simultaneously.

5.1 Stream Length Histogram

The key data structure is a *stream-length likelihood histogram* (LHT), indexed by stream length and storing the cumulative number of stream-length observations. Each time a stream terminates—either from a non-sequential miss or a lifetime expiry—its observed length ℓ is committed to the histogram: buckets 1 through $\min(\ell, L_{\max})$ are each incremented by ℓ. The cumulative weighting means that LHT[k] reflects not just how often streams reached length k, but how much sequential traffic arrived at that depth in total.

To track non-stationary workload phases, the histogram is split into a *current epoch* table $\mathrm{LHT}_{\mathrm{curr}}$ and a *next epoch* accumulator $\mathrm{LHT}_{\mathrm{next}}$. Every E read misses, $\mathrm{LHT}_{\mathrm{next}}$ is promoted to $\mathrm{LHT}_{\mathrm{curr}}$ and cleared for fresh accumulation. This sliding-window design lets the prefetcher adapt within tens of thousands of accesses when a workload phase changes.

5.2 Adaptive Depth Selection

Given a live stream of current length k, the adaptive prefetcher decides how many additional lines to issue by walking $\mathrm{LHT}_{\mathrm{curr}}$:

$$\delta = \max\{n \geq 0 \mid \forall\, i \in [k,\, k{+}n{-}1] : \mathrm{LHT}[i] < 2{\cdot}\mathrm{LHT}[i{+}1]\}$$

capped at $\delta_{\max}$. The factor-of-two threshold encodes a cost model in which one wasted prefetch (using L2 bandwidth with no demand benefit) has the same cost as one demand miss. As long as the histogram says more than half of all streams that reached length i continued to length $i + 1$, it is profitable to issue one more prefetch. When the histogram says most streams end at i, the prefetcher stops.

This rule can be derived from a simple Bayesian decision. Let p_i be the empirical probability that a stream reaching length i extends to $i + 1$. Prefetching is beneficial when:

$$p_i \cdot C_{\mathrm{miss}} > (1 - p_i) \cdot C_{\mathrm{waste}}$$

With $C_{\mathrm{miss}} = C_{\mathrm{waste}}$ (equal cost assumption), the threshold is $p_i > 0.5$, which is exactly $\mathrm{LHT}[i] < 2 \cdot \mathrm{LHT}[i + 1]$.

5.3 Integration into the Pin Tool

A prototype of the adaptive histogram logic exists in `src/stream_buffer.cpp`. Porting it into the Pin tool's `stream_buf` class requires four additions:

1. **Per-stream length counter.** Each `Stream` slot gains a `length` field incremented on every hit. Its value is passed to the histogram commit function when the stream ends.

2. **LHT arrays.** Two `uint64_t` vectors `lht_curr` and `lht_next` of size $L_{\max} + 2$ are added to `stream_buf`. An epoch counter triggers rollover every E read misses.

3. **Adaptive depth call.** The fixed loop bound `depth` in `allocate_stream` and in the stream buffer hit path is replaced by a call to `choose_depth(current_length)`, which walks `lht_curr` according to the inequality above.

4. **Bootstrap prior.** Before the first epoch completes the histogram is empty. A conservative fallback of $\delta = 1$ (one-block lookahead) is used until sufficient observations are available, preventing the prefetcher from issuing no prefetches at all during the warm-up phase.

5.4 Expected Impact

For **libquantum**, the adaptive prefetcher should quickly learn that streams consistently reach the maximum tracked length and issue deep prefetches, matching or exceeding the fixed $d = 8$ result from Figure 2 while wasting negligible bandwidth on mispredictions (since long-stream workloads rarely mispredict).

For **hmmer**, the prefetcher should learn that most streams are short—length 1 or 2—and suppress deep prefetches during pointer-chasing phases while still covering the sequential bursts. The net result is a better accuracy–coverage tradeoff than any fixed-depth policy can achieve: fewer wasted L2 fetches compared to $d = 8$, and lower effective miss count than $d = 1$.

A planned experiment will sweep the epoch size E and maximum tracked length $L_{\max}$ to characterize sensitivity to these tuning parameters and identify an operating point that generalizes across both workloads.

6 Conclusion

This work implemented and evaluated Jouppi's sequential stream buffer inside a Pin-based cache simulator. The results validate the stream buffer's design premise while exposing a sharp dependence on access pattern regularity.

For libquantum, a single prefetch line ($d = 1$) reduces effective L1D misses by 90%—from 23.4M to 2.36M—while adding only 10% more L2 traffic. Deeper buffers provide no further reduction: the effective miss curve is flat beyond $d = 1$, confirming that libquantum's sequential streams are short and regular enough that one prefetched line always arrives in time.

For hmmer, the stream buffer achieves a hit rate of essentially zero at all depths. Its pointer-chasing pattern means every L1D miss allocates a new stream that is immediately discarded, wasting d L2 prefetches with no demand benefit. At $d = 8$, hmmer generates $9\times$ the baseline L2 traffic with no reduction in effective misses.

These results make a clear architectural argument for adaptive depth control. The natural continuation is an online stream-length likelihood histogram that suppresses prefetching when streams are consistently short and issues deep prefetches only when they are consistently long. A prototype of this logic exists in `src/stream_buffer.cpp`; integrating its histogram and depth-selection function into the Pin tool's `stream_buf` class is the primary remaining task.

References

[1] N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," in *Proc. 17th Annual Int'l Symp. on Computer Architecture (ISCA)*, pp. 364–373, May 1990.

[2] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, "Pin: Building customized program analysis tools with dynamic instrumentation," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 190–200, 2005.